

[VN] SOLID là gì và tại sao nó quan trọng trong thiết kế phần mềm?

[EN] What is SOLID and why is it important in software design?

[VN] SOLID là tập hợp 5 nguyên tắc thiết kế phần mềm gồm Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, và Dependency Inversion. Các nguyên tắc này giúp code dễ hiểu, dễ mở rộng, dễ test và dễ bảo trì. Khi tuân theo SOLID, mỗi phần của hệ thống có trách nhiệm rõ ràng, các thành phần ít phụ thuộc chặt vào nhau nên có thể thay đổi hoặc mở rộng mà không làm hỏng code hiện có. Điều này đặc biệt quan trọng trong các hệ thống lớn hoặc microservices vì nó giúp giảm bug, tăng khả năng tái sử dụng và làm cho hệ thống linh hoạt hơn khi yêu cầu thay đổi.

[EN] SOLID is a set of five software design principles: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion. These principles help make code easier to understand, extend, test, and maintain. When a system follows SOLID, each component has a clear responsibility and components are loosely coupled, so changes in one part do not break other parts. This is very important in large systems or microservices because it reduces bugs, improves reusability, and makes the system more flexible when requirements change.

[VN] Single Responsibility Principle (SRP) là gì? Cho ví dụ trong service design.

[EN] What is the Single Responsibility Principle (SRP)? Give an example in service design.

[VN] Single Responsibility Principle (SRP) nói rằng một class, module hoặc service chỉ nên có một trách nhiệm chính và chỉ nên có một lý do để thay đổi. Điều này giúp code dễ hiểu, dễ test và dễ bảo trì vì mỗi thành phần chỉ tập trung vào một nhiệm vụ cụ thể. Ví dụ trong service design: nếu có một UserService vừa xử lý logic người dùng, vừa gửi email, vừa ghi log thì nó vi phạm SRP. Thay vào đó nên tách thành UserService để xử lý logic người dùng, EmailService để gửi email và LoggingService để ghi log. Khi đó nếu thay đổi cách gửi email thì chỉ cần sửa EmailService mà không ảnh hưởng các phần khác.

[EN] The Single Responsibility Principle (SRP) states that a class, module, or service should have one main responsibility and one reason to change. This makes the code easier to understand, test, and maintain because each component focuses on a single task. For example in service design, if a UserService handles user logic, sends emails, and writes logs, it breaks SRP. A better design is to separate them into UserService for user logic, EmailService for sending emails, and LoggingService for logging. If the email logic changes, only EmailService needs to be updated without affecting other parts of the system.

[VN] Open/Closed Principle (OCP) là gì? Làm sao để mở rộng code mà không sửa code cũ?

[EN] What is the Open/Closed Principle (OCP)? How can you extend code without modifying the existing code?

[VN] Open/Closed Principle (OCP) nói rằng phần mềm nên mở để mở rộng (open for extension) nhưng đóng để sửa đổi (closed for modification). Điều này nghĩa là khi cần thêm tính năng mới, chúng ta nên thêm code mới thay vì sửa code đã có, để tránh gây lỗi cho hệ thống đang chạy ổn định. Cách phổ biến để áp dụng OCP là dùng interface, abstraction hoặc design pattern như Strategy hoặc Factory. Ví dụ trong service design: nếu có hệ thống thanh toán với PaymentService, thay vì sửa code mỗi khi thêm phương thức mới như PayPal hoặc Stripe, ta định nghĩa PaymentInterface và tạo các class như StripePayment hoặc PaypalPayment. Khi cần thêm phương thức mới, chỉ cần tạo class mới mà không cần sửa code cũ.

[EN] The Open/Closed Principle (OCP) states that software should be open for extension but closed for modification. This means when we add new features, we should add new code instead of changing existing code, which helps avoid breaking stable parts of the system. A common way to follow OCP is using interfaces, abstraction, or design patterns like Strategy or Factory. For example in a payment system, instead of modifying PaymentService every time we add a new method like PayPal or Stripe, we define a PaymentInterface and create classes such as StripePayment or PaypalPayment. When adding a new payment method, we just create a new class without changing existing code.

[VN] Liskov Substitution Principle (LSP) nghĩa là gì? Khi nào subclass có thể gây bug?

[EN] What is the Liskov Substitution Principle (LSP)? When can a subclass cause bugs?

[VN] Liskov Substitution Principle (LSP) nói rằng một class con (subclass) phải có thể thay thế class cha (base class) mà không làm thay đổi hành vi đúng của chương trình. Nói cách khác, nếu code đang dùng một class cha, thì khi thay bằng class con chương trình vẫn phải chạy đúng như mong đợi. Subclass có thể gây bug khi nó thay đổi hành vi gốc của class cha, ví dụ: method ở class cha cho phép một hành động nhưng class con lại throw error hoặc không hỗ trợ, hoặc làm thay đổi logic quan trọng. Một ví dụ phổ biến là Rectangle và Square: nếu Square kế thừa Rectangle nhưng việc thay đổi chiều rộng làm thay đổi cả chiều cao, thì nó có thể phá vỡ logic đang mong đợi của Rectangle.

[EN] The Liskov Substitution Principle (LSP) states that a subclass should be able to replace its base class without breaking the correct behavior of the program. In other words, if code works with a base class, it should still work correctly when a subclass is used

instead. A subclass can cause bugs when it changes the expected behavior of the base class, for example when a method in the base class allows an operation but the subclass throws an error or does not support it, or changes important logic. A common example is Rectangle and Square: if Square extends Rectangle but changing the width also changes the height, it can break logic that expects normal Rectangle behavior.

[VN] Interface Segregation Principle (ISP) giải quyết vấn đề gì trong thiết kế API?

[EN] What problem does the Interface Segregation Principle (ISP) solve in API design?

[VN] Interface Segregation Principle (ISP) nói rằng một class không nên bị ép phải implement những method mà nó không cần. Thay vì tạo một interface lớn với nhiều chức năng, ta nên chia thành các interface nhỏ và chuyên biệt để mỗi class chỉ implement phần nó thực sự sử dụng. Trong thiết kế API, nguyên tắc này giúp giảm coupling, dễ bảo trì và dễ mở rộng, vì client chỉ phụ thuộc vào các interface nhỏ cần thiết thay vì một interface lớn chứa nhiều method không liên quan. Ví dụ, thay vì một interface UserService có cả createUser, sendEmail, generateReport, ta có thể tách thành các interface nhỏ như UserManager, NotificationService, và ReportService.

[EN] The Interface Segregation Principle (ISP) states that a class should not be forced to implement methods it does not need. Instead of creating one large interface with many responsibilities, we should split it into smaller, more specific interfaces so each class only implements what it actually uses. In API design, this principle helps reduce coupling and improve maintainability and extensibility, because clients depend only on the interfaces they need rather than a large interface with unrelated methods. For example, instead of a single UserService interface containing createUser, sendEmail, and generateReport, we can separate them into smaller interfaces such as UserManager, NotificationService, and ReportService.

[VN] Dependency Inversion Principle (DIP) khác gì dependency injection?

[EN] How is the Dependency Inversion Principle (DIP) different from dependency injection?

[VN] Dependency Inversion Principle (DIP) là một nguyên tắc thiết kế trong SOLID nói rằng module cấp cao không nên phụ thuộc trực tiếp vào module cấp thấp, mà cả hai nên phụ thuộc vào abstraction (interface hoặc abstract class). Điều này giúp hệ thống dễ mở rộng và thay đổi mà không cần sửa code chính. Dependency Injection (DI) là một kỹ thuật để thực hiện DIP, trong đó dependency được truyền từ bên ngoài vào (qua constructor, function parameter, hoặc framework) thay vì class tự tạo ra dependency. Nói ngắn gọn, DIP là nguyên tắc thiết kế, còn DI là cách triển khai nguyên tắc đó trong code.

[EN] The Dependency Inversion Principle (DIP) is a SOLID design principle that states high-level modules should not depend directly on low-level modules; both should depend on abstractions (interfaces or abstract classes). This makes the system easier to extend and change without modifying core code. Dependency Injection (DI) is a technique used to implement DIP, where dependencies are provided from outside (via constructor, function parameters, or a framework) instead of being created inside the class. In short, DIP is the design principle, while DI is a practical way to apply that principle in code.

[VN] Làm sao áp dụng SOLID khi thiết kế microservice?

[EN] How can SOLID be applied when designing microservices?

[VN] Khi thiết kế microservice, có thể áp dụng SOLID để làm cho hệ thống dễ mở rộng và dễ bảo trì. Single Responsibility nghĩa là mỗi service chỉ nên có một trách nhiệm chính, ví dụ service quản lý user không nên xử lý payment. Open/Closed giúp mở rộng chức năng bằng cách thêm module hoặc service mới thay vì sửa code cũ. Liskov Substitution đảm bảo các implementation có thể thay thế nhau mà không làm hỏng logic, ví dụ thay đổi provider gửi email. Interface Segregation nghĩa là service chỉ nên expose API nhỏ và rõ ràng cho từng mục đích. Dependency Inversion khuyến khích service phụ thuộc vào abstraction (interface, contract API) thay vì implementation cụ thể, giúp dễ thay đổi database, message queue hoặc external service.

[EN] When designing microservices, SOLID principles help make the system easier to scale and maintain. Single Responsibility means each service should have one main responsibility, for example a user service should not handle payments. Open/Closed allows adding new features by extending modules or services instead of modifying existing code. Liskov Substitution ensures different implementations can replace each other without breaking logic, such as switching an email provider. Interface Segregation means services should expose small and clear APIs for specific purposes. Dependency Inversion encourages services to depend on abstractions (interfaces or API contracts) rather than concrete implementations, making it easier to change databases, message queues, or external services.

[VN] SRP ảnh hưởng thế nào đến việc chia service trong microservices?

[EN] How does SRP affect service decomposition in microservices?

[VN] Single Responsibility Principle (SRP) ảnh hưởng trực tiếp đến cách chia service trong microservices. Theo SRP, mỗi service nên có một trách nhiệm chính và một lý do để thay đổi. Điều này giúp service nhỏ, rõ ràng và dễ bảo trì. Ví dụ, User Service chỉ quản lý thông tin người dùng, Order Service xử lý đơn hàng, và Payment Service xử lý thanh toán. Khi mỗi service tập trung vào một nhiệm vụ, hệ thống sẽ dễ scale, dễ deploy độc lập và

giảm rủi ro khi thay đổi code. Nếu một service làm quá nhiều việc, nó sẽ trở nên khó bảo trì và phá vỡ nguyên tắc microservices.

[EN] The Single Responsibility Principle (SRP) directly affects how services are divided in a microservices architecture. According to SRP, each service should have one main responsibility and one reason to change. This makes services smaller, clearer, and easier to maintain. For example, a User Service manages user data, an Order Service handles orders, and a Payment Service processes payments. When each service focuses on a single responsibility, the system becomes easier to scale, deploy independently, and maintain. If a service handles too many responsibilities, it becomes harder to manage and breaks the idea of microservices.

[VN] DIP giúp test hệ thống dễ hơn như thế nào?

[EN] How does the Dependency Inversion Principle (DIP) make system testing easier?

[VN] Dependency Inversion Principle (DIP) giúp việc test hệ thống dễ hơn vì code phụ thuộc vào abstraction (interface) thay vì implementation cụ thể. Khi viết test, ta có thể thay thế các dependency thật như database, API bên ngoài, hoặc message queue bằng mock hoặc fake implementation. Điều này giúp test chạy nhanh hơn, không phụ thuộc vào hệ thống bên ngoài và dễ kiểm soát dữ liệu test. Nhờ đó, developer có thể test logic của service một cách độc lập và phát hiện lỗi sớm.

[EN] The Dependency Inversion Principle (DIP) makes systems easier to test because the code depends on abstractions (interfaces) instead of concrete implementations. During testing, real dependencies such as databases, external APIs, or message queues can be replaced with mock or fake implementations. This allows tests to run faster, removes dependency on external systems, and makes test data easier to control. As a result, developers can test the service logic independently and detect bugs earlier.

[VN] ISP giúp thiết kế API client tốt hơn ra sao?

[EN] How does the Interface Segregation Principle (ISP) improve API client design?

[VN] Interface Segregation Principle (ISP) giúp thiết kế API client tốt hơn bằng cách chia interface lớn thành nhiều interface nhỏ và chuyên biệt. Điều này đảm bảo mỗi client chỉ cần phụ thuộc vào những phương thức mà nó thực sự sử dụng, thay vì phải implement hoặc biết những chức năng không cần thiết. Nhờ vậy, code trở nên rõ ràng hơn, dễ bảo trì hơn và giảm rủi ro khi thay đổi API. Ví dụ, thay vì một interface lớn cho user service, ta có thể tách thành các interface nhỏ như UserReader, UserWriter, hoặc UserAuthenticator, giúp các client chỉ dùng đúng phần cần thiết.

[EN] The Interface Segregation Principle (ISP) improves API client design by splitting a large interface into smaller and more specific interfaces. This ensures that each client depends only on the methods it actually uses, instead of being forced to implement or know unnecessary functions. As a result, the code becomes clearer, easier to maintain, and less risky when the API changes. For example, instead of one large interface for a user service, we can split it into smaller interfaces like `UserReader`, `UserWriter`, or `UserAuthenticator`, so clients only use the parts they need.

[VN] Khi nào việc áp dụng SOLID quá mức gây over-engineering?

[EN] When does applying SOLID excessively lead to over-engineering?

[VN] Việc áp dụng SOLID quá mức có thể gây over-engineering khi hệ thống còn nhỏ hoặc logic đơn giản nhưng lại bị chia thành quá nhiều lớp, interface và abstraction. Điều này làm code trở nên phức tạp, khó đọc và khó bảo trì hơn so với lợi ích thực tế. Ví dụ, tạo nhiều interface và layer chỉ để tuân thủ nguyên tắc, trong khi hệ thống chưa có nhu cầu mở rộng hoặc thay đổi. Trong thực tế, SOLID nên được áp dụng khi hệ thống bắt đầu lớn hơn, có nhiều dependency hoặc cần khả năng mở rộng và test tốt hơn.

[EN] Applying SOLID too strictly can lead to over-engineering when the system is small or the logic is simple but the code is split into too many classes, interfaces, and abstractions. This can make the code more complex, harder to read, and harder to maintain without real benefits. For example, creating many interfaces and layers just to follow the principle even though the system does not yet need extensibility or flexibility. In practice, SOLID should be applied when the system becomes larger, has many dependencies, or requires better scalability and testability.

[VN] Trong hệ thống lớn, làm sao giữ codebase vẫn tuân theo SOLID?

[EN] In large systems, how can the codebase be kept compliant with SOLID?

[VN] Trong hệ thống lớn, để giữ codebase vẫn tuân theo SOLID, cần thiết kế kiến trúc rõ ràng và chia hệ thống thành các module hoặc service có trách nhiệm riêng. Mỗi module nên có interface rõ ràng, tránh phụ thuộc trực tiếp vào implementation cụ thể mà phụ thuộc vào abstraction. Code cần được review thường xuyên để đảm bảo mỗi class có một trách nhiệm chính, dễ mở rộng mà không phải sửa code cũ. Ngoài ra, sử dụng dependency injection, viết test tốt và refactor định kỳ giúp hệ thống duy trì cấu trúc sạch và tuân theo SOLID khi dự án phát triển lớn hơn.

[EN] In large systems, to keep the codebase following SOLID, it is important to design a clear architecture and split the system into modules or services with clear responsibilities. Each module should have a clear interface and avoid depending on concrete implementations, instead depend on abstractions. Regular code reviews help ensure that

each class has a single responsibility and that the code can be extended without modifying existing logic. In addition, using dependency injection, writing good tests, and doing periodic refactoring help maintain a clean structure and keep the system aligned with SOLID as the project grows.

[VN] SOLID có áp dụng được cho distributed system không? Nếu có thì như thế nào?

[EN] Can SOLID be applied to distributed systems? If so, how?

[VN] SOLID vẫn có thể áp dụng trong distributed system, nhưng ở mức thiết kế service và module thay vì chỉ trong class. Ví dụ, Single Responsibility Principle giúp mỗi microservice chỉ xử lý một domain hoặc một business capability rõ ràng. Open/Closed Principle cho phép mở rộng hệ thống bằng cách thêm service mới hoặc version API mới mà không cần thay đổi service cũ. Liskov Substitution Principle đảm bảo các service implementation khác nhau (ví dụ nhiều provider hoặc adapter) có thể thay thế nhau mà không làm hỏng logic hệ thống. Interface Segregation Principle giúp thiết kế API nhỏ gọn, mỗi client chỉ gọi đúng endpoint mình cần. Dependency Inversion Principle giúp service phụ thuộc vào interface hoặc contract (như API contract, message schema) thay vì phụ thuộc trực tiếp vào implementation của service khác. Nhờ vậy hệ thống distributed dễ mở rộng, dễ test và ít bị coupling giữa các service.

[EN] SOLID can also be applied in distributed systems, but mainly at the service and module design level instead of only at the class level. For example, the Single Responsibility Principle means each microservice should handle one clear domain or business capability. Open/Closed Principle allows the system to be extended by adding new services or new API versions without modifying existing services. Liskov Substitution Principle ensures different service implementations (such as multiple providers or adapters) can replace each other without breaking system behavior. Interface Segregation Principle helps design small and focused APIs so each client only calls what it needs. Dependency Inversion Principle means services depend on interfaces or contracts (like API contracts or message schemas) rather than concrete implementations of other services. This makes distributed systems easier to scale, test, and maintain with lower coupling between services.

[VN] Khi thiết kế domain service lớn, làm sao tránh vi phạm SRP?

[EN] When designing a large domain service, how can SRP violations be avoided?

[VN] Khi thiết kế một domain service lớn, để tránh vi phạm Single Responsibility Principle (SRP), cần chia logic theo business responsibility rõ ràng thay vì gom nhiều nhiệm vụ vào một service. Một cách phổ biến là tách các phần như validation, business rules, data access, và external integration thành các module hoặc class riêng (ví dụ: validator, repository, adapter). Domain service nên chỉ tập trung điều phối business logic

chính của use case. Ngoài ra có thể áp dụng các pattern như service layer, domain service, và use-case handler để giữ mỗi thành phần chỉ có một lý do để thay đổi. Cách này giúp code dễ đọc, dễ test và dễ mở rộng khi hệ thống lớn lên.

[EN] When designing a large domain service, to avoid violating the Single Responsibility Principle (SRP), the logic should be separated by clear business responsibilities instead of putting many tasks into one service. A common approach is to split parts such as validation, business rules, data access, and external integrations into separate modules or classes (for example: validator, repository, adapter). The domain service should mainly coordinate the core business logic of the use case. In addition, patterns like service layer, domain service, and use-case handlers can help ensure each component has only one reason to change. This approach makes the code easier to read, test, and extend as the system grows.

[VN] Trong microservices architecture, DIP có thể áp dụng ở level service communication như thế nào?

[EN] In microservices architecture, how can DIP be applied at the service communication level?

[VN] Trong microservices architecture, Dependency Inversion Principle (DIP) có thể áp dụng ở level service communication bằng cách để service phụ thuộc vào abstraction thay vì phụ thuộc trực tiếp vào implementation của service khác. Ví dụ, thay vì gọi trực tiếp HTTP API của một service cụ thể, hệ thống có thể định nghĩa một interface hoặc contract (API schema, message schema, hoặc client interface). Service chỉ biết đến abstraction đó, còn implementation có thể thay đổi (REST, gRPC, message queue, mock service khi test) mà không ảnh hưởng đến business logic. Cách này giúp giảm coupling giữa các service, tăng khả năng thay thế, và làm cho việc testing hoặc thay đổi communication mechanism dễ dàng hơn.

[EN] In microservices architecture, the Dependency Inversion Principle (DIP) can be applied at the service communication level by making services depend on abstractions instead of concrete implementations of other services. For example, instead of directly calling a specific HTTP API of another service, the system can define an interface or contract (such as an API schema, message schema, or client interface). The service only depends on that abstraction, while the actual implementation can change (REST, gRPC, message queue, or a mock service for testing) without affecting the business logic. This approach reduces coupling between services, improves flexibility, and makes testing or changing the communication mechanism easier.

[VN] Design pattern là gì và tại sao nó quan trọng trong system design?

[EN] What is a design pattern and why is it important in system design?

[VN] Design pattern là các giải pháp thiết kế đã được kiểm chứng để giải quyết những vấn đề phổ biến trong phát triển phần mềm. Nó không phải là code cụ thể mà là một cách tổ chức code và trách nhiệm giữa các thành phần trong hệ thống. Trong system design, design pattern quan trọng vì nó giúp kiến trúc hệ thống rõ ràng hơn, giảm độ phức tạp, tăng khả năng tái sử dụng và bảo trì code, đồng thời giúp các developer dễ hiểu cấu trúc hệ thống vì họ đã quen với các pattern phổ biến như Singleton, Factory, Observer hoặc Strategy.

[EN] A design pattern is a proven design solution for common problems in software development. It is not a specific piece of code, but a general way to organize code and responsibilities between components in a system. In system design, design patterns are important because they make the architecture clearer, reduce complexity, improve code reuse and maintainability, and help developers understand the system structure more easily since many developers are already familiar with common patterns such as Singleton, Factory, Observer, or Strategy.

[VN] Singleton pattern dùng khi nào? Khi nào không nên dùng?

[EN] When should the Singleton pattern be used? When should it not be used?

[VN] Singleton pattern là một design pattern đảm bảo rằng chỉ có một instance của một class được tạo ra trong toàn bộ ứng dụng và cung cấp một điểm truy cập toàn cục đến instance đó. Nó thường được dùng khi hệ thống cần một tài nguyên dùng chung như configuration manager, logger, cache hoặc connection pool để tránh tạo nhiều instance không cần thiết. Tuy nhiên, không nên dùng Singleton khi nó làm tăng sự phụ thuộc toàn cục (global state), gây khó khăn cho việc test, hoặc khi ứng dụng cần nhiều instance độc lập, vì nó có thể làm code khó mở rộng và khó bảo trì.

[EN] The Singleton pattern is a design pattern that ensures only one instance of a class is created in the whole application and provides a global access point to that instance. It is commonly used when the system needs a shared resource such as a configuration manager, logger, cache, or connection pool to avoid creating multiple unnecessary instances. However, it should not be used when it introduces too much global state, makes testing harder, or when the application needs multiple independent instances, because it can reduce flexibility and make the code harder to maintain.

[VN] Factory pattern giúp giải quyết vấn đề gì?

[EN] What problem does the Factory pattern solve?

[VN] Factory pattern là một design pattern giúp tách việc tạo object khỏi logic sử dụng object. Thay vì tạo object trực tiếp bằng new hoặc gọi constructor ở nhiều nơi, hệ thống dùng một factory để quyết định loại object nào cần tạo dựa trên điều kiện hoặc cấu hình.

Cách này giúp code linh hoạt hơn, dễ mở rộng theo nguyên tắc Open/Closed, và giảm sự phụ thuộc giữa các module. Nó thường được dùng khi có nhiều loại object cùng interface hoặc khi logic tạo object phức tạp và có thể thay đổi theo thời gian.

[EN] The Factory pattern is a design pattern that separates object creation from the logic that uses the object. Instead of creating objects directly with new or calling constructors in many places, the system uses a factory to decide which type of object should be created based on conditions or configuration. This makes the code more flexible, easier to extend following the Open/Closed principle, and reduces coupling between modules. It is commonly used when there are multiple types of objects with the same interface or when object creation logic is complex and may change over time.

[VN] Builder pattern khác gì constructor truyền nhiều tham số?

[EN] How is the Builder pattern different from a constructor with many parameters?

[VN] Builder pattern dùng để tạo object từng bước và giúp code dễ đọc khi object có nhiều thuộc tính tùy chọn. Thay vì truyền rất nhiều tham số vào constructor (dễ gây khó hiểu, sai thứ tự tham số và khó mở rộng), builder cho phép thiết lập từng thuộc tính bằng các method riêng rồi mới gọi build() để tạo object. Cách này giúp code rõ ràng hơn, dễ bảo trì và tránh lỗi khi có nhiều tham số hoặc nhiều cấu hình khác nhau.

[EN] The Builder pattern is used to create an object step by step and makes the code easier to read when the object has many optional fields. Instead of passing many parameters into a constructor (which can be confusing, error-prone, and hard to extend), a builder allows setting each property with separate methods and then calling build() to create the object. This approach makes the code clearer, easier to maintain, and safer when there are many parameters or different configurations.

[VN] Strategy pattern dùng khi nào trong business logic?

[EN] When is the Strategy pattern used in business logic?

[VN] Strategy pattern được dùng khi một business logic có nhiều cách thực hiện khác nhau và có thể thay đổi tùy theo điều kiện hoặc cấu hình. Thay vì dùng nhiều câu lệnh if/else hoặc switch, ta tách mỗi cách xử lý thành một strategy riêng và chọn strategy phù hợp khi chạy chương trình. Điều này giúp code dễ mở rộng, dễ test và tuân theo nguyên tắc Open/Closed vì có thể thêm logic mới mà không cần sửa code cũ.

[EN] The Strategy pattern is used when a business logic has multiple ways to perform the same task and the method can change based on conditions or configuration. Instead of using many if/else or switch statements, each algorithm is placed in a separate strategy and the correct one is selected at runtime. This makes the code easier to extend, easier to test,

and follows the Open/Closed principle because new behaviors can be added without modifying existing code.

[VN] Dependency Injection pattern giúp code dễ test như thế nào?

[EN] How does the Dependency Injection pattern make code easier to test?

[VN] Dependency Injection (DI) là một design pattern trong đó các dependency của một class được truyền từ bên ngoài vào thay vì class tự tạo chúng. Điều này giúp tách rời business logic khỏi implementation cụ thể, ví dụ như database, API client hoặc service khác. Khi viết test, ta có thể dễ dàng thay thế dependency thật bằng mock hoặc stub để kiểm soát dữ liệu và hành vi, từ đó test nhanh hơn, ổn định hơn và không phụ thuộc vào hệ thống bên ngoài như database hoặc network.

[EN] Dependency Injection (DI) is a design pattern where the dependencies of a class are provided from the outside instead of being created inside the class. This separates business logic from specific implementations such as databases, API clients, or other services. When writing tests, we can easily replace real dependencies with mocks or stubs to control data and behavior. This makes tests faster, more stable, and independent from external systems like databases or networks.

[VN] Adapter pattern dùng khi tích hợp hệ thống legacy như thế nào?

[EN] How is the Adapter pattern used when integrating legacy systems?

[VN] Adapter pattern được dùng khi cần tích hợp hệ thống legacy có interface cũ hoặc khác với interface mà hệ thống hiện tại mong đợi. Adapter hoạt động như một lớp trung gian, chuyển đổi interface của hệ thống legacy sang interface chuẩn mà application đang sử dụng. Nhờ vậy, code mới không cần thay đổi nhiều và không phụ thuộc trực tiếp vào implementation cũ, giúp hệ thống dễ bảo trì, dễ mở rộng và giảm rủi ro khi thay đổi hoặc thay thế hệ thống legacy trong tương lai.

[EN] The Adapter pattern is used when integrating a legacy system that has an old or different interface from what the current system expects. The adapter acts as a middle layer that converts the legacy system's interface into the standard interface used by the application. This allows the new code to work without major changes and avoids direct dependency on the old implementation, making the system easier to maintain, extend, and replace the legacy system in the future.

[VN] Facade pattern giúp đơn giản hóa hệ thống lớn ra sao?

[EN] How does the Facade pattern simplify large systems?

[VN] Facade pattern giúp đơn giản hóa hệ thống lớn bằng cách cung cấp một interface đơn giản để truy cập vào nhiều module hoặc subsystem phức tạp phía sau. Thay vì client phải

gọi nhiều class hoặc service khác nhau, client chỉ cần tương tác với một lớp facade duy nhất, và lớp này sẽ điều phối các lời gọi đến các thành phần bên trong. Cách này giúp giảm sự phụ thuộc giữa các module, làm code dễ hiểu, dễ bảo trì và giúp thay đổi implementation bên trong mà không ảnh hưởng đến client.

[EN] The Facade pattern simplifies a large system by providing a simple interface to access many complex modules or subsystems behind it. Instead of the client calling many classes or services directly, the client only interacts with one facade class, and this class coordinates calls to the internal components. This approach reduces coupling between modules, makes the code easier to understand and maintain, and allows internal implementation changes without affecting the client.

[VN] Repository pattern có vai trò gì trong clean architecture?

[EN] What role does the Repository pattern play in clean architecture?

[VN] Repository pattern có vai trò tách business logic khỏi data access logic trong Clean Architecture. Repository hoạt động như một lớp trung gian giữa domain/service và database, cung cấp các method để đọc và ghi dữ liệu mà không để business logic phụ thuộc trực tiếp vào ORM hay database cụ thể. Nhờ đó code dễ test hơn (có thể mock repository), dễ thay đổi database hoặc framework mà không ảnh hưởng đến logic nghiệp vụ, và giúp hệ thống tuân theo nguyên tắc Dependency Inversion.

[EN] The Repository pattern separates business logic from data access logic in Clean Architecture. It acts as a middle layer between the domain/service layer and the database, providing methods to read and write data without letting business logic depend directly on a specific ORM or database. This makes the code easier to test (repositories can be mocked), easier to change the database or framework without affecting business logic, and helps the system follow the Dependency Inversion principle.

I. Fundamentals – Design Principles & Patterns

SOLID là gì và tại sao nó quan trọng trong thiết kế phần mềm?

Single Responsibility Principle(SRP) là gì? Cho ví dụ trong service design.

Open/Closed Principle(OCP) là gì? Làm sao để mở rộng code mà không sửa code cũ?

Liskov Substitution Principle(LSP) nghĩa là gì? Khi nào subclass có thể gây bug?

Interface Segregation Principle(ISP) giải quyết vấn đề gì trong thiết kế API?

Dependency Inversion Principle(DIP) khác gì dependency injection?

Làm sao áp dụng SOLID khi thiết kế microservice?

SRP ảnh hưởng thế nào đến việc chia service trong microservices?

DIP giúp test hệ thống dễ hơn như thế nào?

ISP giúp thiết kế API client tốt hơn ra sao?

Khi nào việc áp dụng SOLID quá mức gây over-engineering?

Trong hệ thống lớn, làm sao giữ codebase vẫn tuân theo SOLID?

SOLID có áp dụng được cho distributed system không? Nếu có thì như thế nào?

Khi thiết kế domain service lớn, làm sao tránh vi phạm SRP?

Trong microservices architecture, DIP có thể áp dụng ở level service communication như thế nào?

Design pattern là gì và tại sao nó quan trọng trong system design?

Singleton pattern dùng khi nào? Khi nào không nên dùng?

Factory pattern giúp giải quyết vấn đề gì?

Builder pattern khác gì constructor truyền nhiều tham số?

Strategy pattern dùng khi nào trong business logic?

Dependency Injection pattern giúp code dễ test như thế nào?

Adapter pattern dùng khi tích hợp hệ thống legacy như thế nào?

Facade pattern giúp đơn giản hóa hệ thống lớn ra sao?

Repository pattern có vai trò gì trong clean architecture?